

标准 IO 流

`1024*8` 是和 `BufferOutputStream` 一样的缓存大小，并且 更加 高效

```
FileInputStream fis = null;
FileOutputStream fos = null;
try {
    fis = (FileInputStream) body.byteStream();
    fos = new FileOutputStream(filePath);
    byte[] buff = new byte[1024*8];
    int len;
    while ((len=fis.read(buff)) != -1) {
        fos.write(buff,0,len);
    }
    if (callback != null)
        callback.onResponse("200");
} catch (Exception e) {
    e.printStackTrace();
} finally {
    IOUtils.closeIOs(fis, fos);
}
```

关闭流

IO流标准处理代码

1.6及其以前

try { ... } finally { ... }

或者 throws

```
public static void main(String[] args) throws IOException{
    FileInputStream fis = null;
    FileOutputStream fos = null;

    try{
        fis = new FileInputStream("xxx.txt");
        fos = new FileOutputStream("yyy.txt");
        int a;
        while((a=fis.read())!=-1){
            fos.write(a);
        }
    }catch (FileNotFoundException e) {
        e.printStackTrace();
    }finally{
        try {
            if(fis!=null)
                fis.close();
        } catch (IOException e) {
            e.printStackTrace();
        }finally{
            try {
                if(fos!=null) // 放到
```

finally语句中，能关一个是一个

```

        fos.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
}
}

```

1.7及其以后

try(...){ ... } + throws 或者 **catch**

try(创建流对象) 中创建的流对象，必须继承了 **AutoCloseable** 这个接口，否则会报错；

如果实现了这个接口，则在 {读写代码} 中的代码执行完毕后，**自动调用close方法**，将流关掉

```

public class Test {
    public static void main(String[] args) {
        try(
            FileInputStream fis = new FileInputStream("moon.jpg");
            FileOutputStream fos = new
FileOutputStream("moon2.jpg");
        ){
            int len;
            while((len=fis.read()) != -1){
                fos.write(len);
            }
        }catch(IOException e) {

```

```
e.printStackTrace();
```

```
}
```

```
}
```

```
}
```

IO 流关闭工具类

注意，这里的 `Arrays.asList()` 转成的List集合，是 `java.util.Arrays$List`，而不是 `List`；

- `java.util.Arrays$List` 和 `List` 都是 `AbstractList` 的子类，`remove/add`等操作在 `AbstractList` 中，默认是 throw `UnsupportedOperationException`，而且不做任何操作；
- `List` 对这些方法进行了重写
- `Arrays$List` 对这些方法没有重写，所以无法使用 `remove/add` 等操作，会抛出 `UnsupportedOperationException` 异常
- 我们需要将 `Arrays$List` 转化成 `List` 集合，再使用

```
public class IOUtils {
```

```
    /**
```

```
     * 关闭IO流
```

```
     * @param closeableList
```

```
     */
```

```
    public static void closeIOs(List<Closeable>
```

```
closeableList) {
```

```
        if (closeableList!=null && closeableList.size()>0) {
```

```
            try {
```

```
        Closeable closeable = closeableList.get(0);
        if (closeable != null)
            closeable.close();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        closeableList.remove(0);
        closeIOs(closeableList);
    }
}

/**
 * 关闭IO流
 * @param closeables
 */
public static void closeIOs(Closeable... closeables) {
    List<Closeable> closeableList =
Arrays.asList(closeables);
    closeIOs(new ArrayList<Closeable>(closeableList));
}

}
```

转换流

流转byte

流转 byte 数组的基本思路，就是通过

`ByteArrayOutputStream` 转 byte 数组（`toByteArray` 方法）

```
/**
 * InputStream转byte[]
 * @param is
 * @return
 */
public static byte[] input2bytes(InputStream is) {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    int len = -1;
    byte[] bytes = new byte[1024*8];
    try {
        while ((len=is.read(bytes)) != -1) {
            baos.write(bytes, 0, len);
        }
        return baos.toByteArray();    // 转byte数组
    } catch (IOException e) {
        e.printStackTrace();
        return new byte[0];
    } finally {
        IOUtils.closeIOs(is, baos);
    }
}
```

IO 流的应用

图片加密

$$a^b^b=a$$

加密：异或^一个数

解密：异或^一个数

```
BufferedInputStream bis = new BufferedInputStream(new
FileInputStream("a.jpg"));
BufferedOutputStream bos = new BufferedOutputStream(new
FileOutputStream("b.jpg"));

int b;
while((b = bis.read()) != -1) {
    bos.write(b ^ 123);
}

bis.close();
bos.close();
```

从 URL 中拷贝图片

Url 图片转为 File、InputStream、Bitmap

Url.openStream() 可以将一个 Url 地址转为 InputStream

同理，也可以将一个 Url 的图片地址转化为 Bitmap 对象

```
URL url = new URL("http://baidu.com/123.jpg");
InputStream is = url.openStream();

FileOutputStream fos = new FileOutputStream("abc.jpg");
int len;
while((len=is.read()) != -1){
    fos.write(len);
}
is.close();
fos.close();
```